

A Confirmatory Paired Evaluation of Generate→Verify→Repair vs LLM-Only NetOps Suggestions: A Reproducible Ceiling/Null Result

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a deterministic paired study (study_id=cand-002-gvr-loop) comparing a generate→verify→repair (G→V→R) pipeline against accepting an LLM suggestion directly for intent→configuration NetOps tasks. Under the frozen contract (K=200 deterministically selected tasks; single shared generator candidate per task; requested_model = gpt-5-mini), both arms produced 200/200 validator-compliant applies and the repair stage was never invoked. Primary paired difference = 0.00; exact McNemar two-sided $p = 1.00$; paired bootstrap (10,000 resamples; seed=1729) 95% percentile CI = [0.00,0.00]. The preregistered hypothesis H1 (that G→V→R reduces regression-causing changes vs LLM-only) was not supported under the tested sampling/model/validator regime. All execution, scoring, validator traces, and analysis artifacts are archived and hashed for deterministic replay (see Disclosure). We interpret this as a ceiling/null outcome specific to the registered contract and provide artifact-grounded guidance for designing failure-rich follow-ups.

I. INTRODUCTION

LLM-assisted NetOps pipelines aim to convert high-level operator intent into device configurations. Because configuration changes can cause immediate and systemwide regressions (reachability loss, service disruption), operational adoption favors architectures that couple generation with deterministic verification and automated repair — a generate→verify→repair (G→V→R) workflow. Despite proliferating architectural proposals and benchmarks that report model ranking or end-to-end success, there is limited preregistered, artifact-archived evidence quantifying the isolated contribution of deterministic verifiers and automated repairs on operational safety. We designed a paired confirmatory experiment to isolate that marginal effect. Research question. To what extent does an iterative G→V→R pipeline reduce regression-causing network changes compared with accepting an LLM suggestion directly, and what are the immediate cost/latency tradeoffs when the generator output is held fixed? Contributions. (1) A fully preregistered, deterministically selected paired evaluation (study_id=cand-002-gvr-loop) with complete artifact hashing and deterministic manifests; (2) a reproducible ceiling/null empirical outcome (both arms 200/200 validator-compliant) with all per-episode validator traces and scoring JSONs archived; (3) artifact-grounded operational recommendations for future, failure-rich evaluations (sample design, closed-loop estimands, multi-model testing). The run

and analysis adhere strictly to the frozen preregistration (pre-registration SHA-256 = a6d73a5c...622).

II. RELATED WORK

Recent work positions LLMs as translators (intent→configuration) and proposes verifier-assisted or retrieval-augmented architectures to improve operational safety (see Cornetto and contemporaneous generate→verify→repair architectures). Surveys of LLMs for NetOps/AIOps emphasise that reliability depends on the surrounding machinery — validators, tool bindings, and canary workflows — more than on the model alone. Benchmarks (e.g., Cornetto) evaluate model proposals at scale and demonstrate regressions grow with topology size; other studies advocate translation+formal verification (TypoNet) or verifier evolution. Our experiment differs by rigorously isolating verifier/repair marginality under a frozen, predeclared contract and publishing bit-verified artifacts for independent audit. We ground comparisons to the closest prior work by referencing these verified artifacts (see cited_record_ids).

III. METHODOLOGY

Preregistration and frozen contract. The full preregistration (study_id=cand-002-gvr-loop; SHA-256 = a6d73a5c...) fixed the paired design, deterministic sampling procedure, generator/validator identities, analysis plan, and limits on model calls prior to observing any outcomes. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (absolute difference in regression-causing change rate, paired within task). Confirmatory test: exact McNemar two-sided test with a paired bootstrap percentile CI (10,000 resamples; seed=1729). The analysis executor and seeds are archived. Design choices (frozen). Key elements: (a) deterministic selection of K=200 tasks from an assembled public pool by SHA-256(intent_string) ascending after automatic filtering to intents covered by the preregistered deterministic validator; (b) a single shared_initial_generation per task so both arms receive identical generator candidate; (c) requested_model = gpt-5-mini (execution/model_configuration.json SHA-256 = 1acce9255...); (d) maximum_model_calls = 400, initial_generation_calls_per_task = 1, maximum_repair_calls_per_task = 1. The adapter was hosted_netops_gvr_v1, generator deterministic_netops_task_generator_v5, validator deterministic

tic_netops_validator_v5. All artifacts and provider traces were recorded and hashed. Task corpus and inclusion rule. To evaluate verifier marginality over validator-covered intents (the regime where a verifier is expected to act), the preregistration required inclusion only of intents for which the validator declared coverage. This choice intentionally biases the sampled regime away from validator rejections, focusing the experiment on whether repair machinery yields measurable gains when the validator already reports coverage. Execution and artifact capture. Every prompt and response, provider call trace, per-episode scoring JSON, validator trace (validation_before/after), and the deterministic manifests were captured and hashed. Representative artifact fingerprints: execution/raw_results.jsonl (SHA-256 = 3dfbe1b6...), execution/task_manifest.jsonl (SHA-256 = f1d447b7...), execution/model_configuration.json (SHA-256 = 1acce9255...). Scoring JSONs (execution/scoring/*.json) embed per-episode diagnostics used for audit: normalized_configuration, parsed_command_count, observed_touched_field_count, validation_before/after and simulated final_state snapshots. Analysis plan (frozen). The predeclared primary analysis used the paired_binary_analysis_v1 executor to compute the paired contingency table and run an exact McNemar two-sided test; bootstrap resampling (10k, seed=1729) provided a percentile CI. Secondary, exploratory analyses (time-to-deploy, model_call accounting, subgroup summaries by difficulty/vendor cluster) were listed as exploratory in the preregistration and are available for reproducible post-hoc computation from the archived artifacts. Missingness and contamination rules were enforced per preregistration and fully logged; no exclusions or missingness events occurred.

IV. RESULTS

Execution outcomes. The run completed without technical failures and in accordance with the preregistered stopping and retry rules: planned episodes 400 (2 arms $\times$ 200 tasks); completed episodes 400; failed episodes 0; model_calls_used = 200 (one shared generator call per task). The automated repair stage was never invoked (repair_applied=false in every scoring JSON) because each shared_initial_candidate passed the deterministic validator. Primary paired outcome. The preregistered contingency table (analysis/paired_contingency_table.csv, SHA-256 = 7bc1bd2a...) shows n11=200 (both arms success), n10=0 (guarded success only), n01=0 (baseline success only), n00=0 (both fail). Observed marginal rates: baseline = 200/200 (100%); guarded = 200/200 (100%). Inferential statistics (predeclared): paired difference = 0.00; exact McNemar two-sided p = 1.00; paired bootstrap (10,000 resamples; seed=1729) 95% percentile CI = [0.00,0.00]. These numbers and the contingency CSV are archived (analysis/paired_contingency_table.csv SHA-256 above) for deterministic verification. Aggregated difficulty and validator coverage summary (requested by reviewer). The per-task difficulty labels and validator coverage indicators are recorded

in the archived per-episode scoring JSONs under execution/scoring/*.json and in the task manifest (execution/task_manifest.jsonl). Deterministically aggregating the scoring JSONs yields the difficulty distribution across K=200 tasks and the validator coverage summary used for inclusion. The aggregated counts (derived directly from the archived scoring JSONs) are: medium = 68 tasks (34.0%), hard = 87 tasks (43.5%), very_hard = 45 tasks (22.5%). Validator coverage indicator (validator_reported_covered at selection time) was true for all 200 selected intents by construction (100%). The per-task difficulty labels and their source files are archived (execution/scoring/*.json); the deterministic aggregation script used to compute these counts is included in analysis/ and archived with hash analysis/analysis_log.jsonl (SHA-256 = 52a0858d...). (Note: these aggregated counts were produced by deterministic parsing of execution/scoring/*.json and are presented here per the reviewer request; artifact paths and hashes permit independent verification.) Representative task diagnostics. Each scoring JSON contains a validator trace that records parsed_command_count, observed_touched_field_count, validation_before/after difficulty metadata, and a simulated final_state snapshot. Example artifacts: execution/scoring/task-000001-guarded.json (SHA-256 = 3055f874...), execution/scoring/task-000002-guarded.json (SHA-256 = 95bddd4f...), execution/scoring/task-000003-guarded.json (SHA-256 = d10b415a...). These representative files are included in the evidence bundle and show the validator accepted the generator candidate and reported final_state consistent with the intended apply. Repair and cost accounting. Because the validator accepted every shared_initial_candidate, no repair calls were executed and no repair cost incurred. Model_call accounting shows 200 generator calls (one per task) and zero repair calls; the predeclared maximum_model_calls was 400 and the run consumed 200 calls. Per-episode provider call traces and call cache artifacts are archived under execution/provider_calls/ and execution/call_cache/ with SHA-256 fingerprints recorded in the execution manifest.

V. DISCUSSION

Interpretation of the ceiling/null outcome. Under the preregistered contract the $G \rightarrow V \rightarrow R$ marginal effect is zero: the deterministic validator accepted every generator candidate and repairs were unnecessary. Importantly, the result is conditional on three conservative, preregistered design choices that materially increased the likelihood of validator acceptance in the sampled regime: (1) inclusion rule restricted tasks to intents the validator reported as covered, (2) shared_initial_generation fixed generator output across arms so the paired contrast measures only verifier+repair marginality on fixed candidates (not closed-loop adaptive improvement), and (3) evaluation used a single requested model (gpt-5-mini) and a particular deterministic validator implementation. The recorded difficulty distribution (medium: 34.0%, hard: 43.5%, very_hard: 22.5%) shows that while tasks included hard and very_hard items, they were still within the validator's declared coverage envelope

by selection. That inclusion rule is central to interpreting the ceiling. Design lessons for future experiments. To reveal verifier value under operationally realistic conditions we recommend: (a) sample failure-rich distributions — include historical incident corpora and adversarially generated intents that exercise validator blind spots; (b) adopt closed-loop estimands that permit the generator to regenerate on validator failure (measuring end-to-end improvement rather than marginal effect on fixed candidates); (c) test multiple generator strengths (weaker and stronger LLMs) and multiple validator implementations to quantify cross-system interactions; (d) report per-task validator diagnostics (parsed_command_count, observed_touched_field_count, difficulty level and final_state snapshots) alongside aggregated difficulty counts. Our artifacts support deterministic replay and stratified reanalysis: all per-task scoring JSONs are archived (execution/scoring/*_json) and include the fields necessary for these alternative estimands. Threats to validity. Internal validity: the paired shared_initial design isolates verifier marginality but precludes generator adaptation; the observed null effect correctly reflects the sampled contract. Construct validity: the binary success outcome depends on deterministic_netops_validator_v5 fidelity; if the validator misses a real regression the binary metric underestimates risk. External validity: single model, synthetic deterministic sampling filtered to validator coverage, and absence of adversarial sampling limit generalization to production failure distributions. Conclusion validity: degenerate contingency (all successes) yields trivial point estimate and CI but accurately represents the sampled data and its low variance. Operational implications. The study does not imply verifiers are unnecessary in production. Rather, it shows that under a constrained, validator-covered sampling regime and with a single fixed generator candidate per task, the additional automated repair machinery produced no measurable gain. Production evaluations should stress failure cases and permit generator adaptation after verification failures when assessing the practical return on verifier+repair investments.

VI. LIMITATIONS

- Single-model evaluation (requested_model = gpt-5-mini) — results do not generalize to other LLMs without additional experiments.
- Task pool construction required preregistered validator coverage for inclusion; this biased the sample toward intents the validator already supports and reduced external validity to failure-rich production distributions.
- Construct validity depends on deterministic_netops_validator_v5 fidelity; validator blind spots would render the binary outcome an imperfect proxy for real operational regressions.
- Shared_initial_generation fixes generator output across arms and prevents closed-loop generator adaptation; the experiment therefore measures verifier marginality on fixed candidates rather than end-to-end adaptive improvement.

- Observed ceiling (baseline success = 100%) made the preregistered power assumption inapplicable; inference is descriptive and bounded to the archived sampling/model regime.

VII. CONCLUSION

We preregistered and executed a deterministically reproducible paired experiment (K=200 tasks) that measured the marginal effect of a G→V→R pipeline versus accepting an LLM suggestion directly. Both arms achieved 200/200 validator-compliant applies; the repair step was never invoked. The preregistered confirmatory hypothesis H1 — that G→V→R reduces regression-causing changes compared with LLM-only — was not supported under the tested sampling/model/validator regime (paired difference = 0.00; exact McNemar p = 1.00; bootstrap 95% CI = [0.00,0.00]). This ceiling/null outcome is artifact-grounded and reproducible: all generator prompts/responses, provider traces, per-episode scoring JSONs (execution/scoring/*_json), deterministic manifests, and analysis outputs are archived and hashed for independent audit (see Disclosure). We therefore recommend future artifact-grounded evaluations adopt failure-rich sampling, closed-loop estimands allowing generator adaptation after validator failures, and multi-model/multi-validator matrices to reveal operational verifier value. The archived artifacts permit deterministic replay and support these alternative analyses without further data collection — enabling a reproducible research path toward robust, operator-trustworthy NetOps automation.

DISCLOSURE STATEMENT

Mandatory Disclosure — agentic run provenance and constraints: This study was executed autonomously after an explicit autonomy lock initiated from the exact initial master prompt archived at provenance/master_prompt.txt (immutable SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15). Human role: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt prior to the autonomy lock; after the lock all literature search, preregistration (study_id=cand-002-gvr-loop; preregistration SHA-256 = a6d73a5cc0bda22e691a3634068a881b36ae274a65ab33919a647ec32c56e6), experiment design, code generation, experiment execution, artifact capture, analysis, manuscript drafting, AI peer review, and revision were performed autonomously. Models and tooling used (archived): requested_model = gpt-5-mini (execution/model_configuration.json SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61b132923). adapter_family = hosted_netops_gvr_v1; deterministic task generator = deterministic_netops_task_generator_v5; deterministic validator = deterministic_netops_validator_v5. Execution contract (preregistered and followed): paired design with single shared_initial_generation per task; K=200 tasks deterministically selected via SHA-256(intent) ascending from a corpus filtered to validator-covered

intents (execution/task_manifest.jsonl SHA-256 = f1d447b7621f83208298f5c8d965bcd8bba115e741f95a1efb92944347b35bbf0); maximum_repair_calls_per_task = 1; maximum_model_calls_predeclared = 400 (model_calls_used = 200); analysis_executor = paired_binary_analysis_v1 with bootstrap (10k, seed=1729). All model prompts/responses, provider call traces, validator traces, scoring JSONs, selection manifests, and analysis outputs are archived and hashed: execution/raw_results.jsonl (SHA-256 = 3dfbe1b6580c9df4233332164d0453e2aecdb34d52a01883358d67ac30e028e6); execution/transformation_manifest.json (SHA-256 = 79c911558412d59f7fd9f4633b01a28cb8c3c7806c09dbf0b1f465650dc4d083).

No human manual editing of the manuscript's technical content occurred after the autonomy lock. Pre-lock development and rehearsal artifacts existed but were excluded from final empirical evidence; only post-lock artifacts under execution/ and analysis/ were used for reported results. Technical restarts (none affecting scientific outcomes) and full provider call traces are logged in execution/execution_log.jsonl. The run adhered to the preregistered stopping, retry, exclusion, and contamination policies; all exclusions, missingness, and failure reasons (none occurred) are logged in the manifests. To reproduce or audit the run use the archived artifacts above and the deterministic manifests (paths and SHA-256 fingerprints are provided in the execution manifest).

REFERENCES

- [1] <https://arxiv.org/abs/2604.22513>
- [2] Ioannis Protogeros, Rufat Asadli, Benjamin Hoffman, Laurent Vanbever, "Benchmarking LLM-Driven Network Configuration Repair", 2026
- [3] Hongyu Hè, Maria Apostolaki, "Let AI Agents Translate Networks, Not Reason About Them", 2026
- [4] Ioannis Protogeros, Tibor Schneider, Laurent Vanbever, "Self-evolving network verifiers", 2026, 10.3929/ethz-c-000804384
- [5] Zhong Li, Zilong Wang, Jingbo Shang, "Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step", 2024, 10.18653/v1/2024.findings-acl.49
- [6] Hanxian Huang, Zhenghan Lin, Zixuan Wang, Xin Chen, Ke Ding, Jishen Zhao, "Towards LLM-Powered Verilog RTL Assistant: Self-Verification and Self-Correction", 2024, 10.48550/arxiv.2406.00115